[Paper Title]

Anonymous Author(s)
Affiliation
Address
email

Abstract

Long-horizon robotic manipulation is non-Markovian: counting events, recovering object identity under occlusion, and reproducing demonstrated motions all require retaining many seconds of visual history while the policy runs at control rate within a fixed token budget. Recent benchmark evidence [Dai et al., 2026] shows that perceptual memory representations dominate motion-centric and time-sensitive tasks, but its concrete instantiations — uniform frame sampling and temporal-saliency-agnostic token dropping — are content-blind and degrade sharply as the memory budget tightens, while symbolic / language-subgoal memory cannot adequately represent low-level continuous motion. We introduce a robot-specialized perceptual memory that decouples *what* the robot saw from *how* things moved between observations: a CLIP-derivative keyframe selector with delta-based per-keyframe TokenDrop carries sparse spatial evidence at semantic transitions, and a learned MotionEncoder, pretrained on unlabeled robot video by JEPA-style [Assran et al., 2023] latent future-token prediction, summarizes each short window into a single compact motion token. The combined sequence is exposed to the $\pi_{0.5}$ backbone through a fixed-budget memory interface compatible with all three Memory-as-Context, -Modulator, and -Expert integration mechanisms used in the MME-VLA suite [Dai et al., 2026], enabling apples-to-apples comparison against uniform-frame-sampling and token-dropping baselines at matched memory budgets. Full quantitative evaluation on RoboMME is *in progress* (Sec. 4).

1 Introduction

Open-world robotic manipulation is rarely Markovian. A household robot that returns books to the right shelves must remember where each book originally came from; a robot asked to "press the button *exactly* when the moving cube reaches the target a specified number of times" must remember an event count and an event ordering; a robot reproducing a demonstrated trajectory must replay motion patterns it observed seconds earlier [Dai et al., 2026]. Modern vision-language-action (VLA) policies run at control rate, on a transformer with a fixed input window, so the central design question for memory-augmented manipulation is not *whether* to retain history but *which projection of it* fits inside that window without losing decision-relevant signal.

Two ends of the design spectrum have been explored, and each is unsatisfactory. *Symbolic* memory [Bu et al., 2025, Sridhar et al., 2025, Gemini Robotics Team et al., 2025] discretizes history into language-based subgoals or natural-language plans; this is cheap and human-interpretable, but recent controlled evaluation [Dai et al., 2026] shows that language under-specifies continuous motion and the resulting policies struggle on motion-centric and precise visuomotor tasks even when given oracle subgoals. *Perceptual* memory keeps raw visual tokens [Jang et al., 2025, Koo et al., 2025, Fang et al., 2025, Shi et al., 2026, Physical Intelligence, 2026a], and is the empirically stronger family on motion-centric and long-horizon tasks [Dai et al., 2026]; but the two perceptual instantiations evaluated head-to-head in that study — uniform frame sampling and temporal-saliency-agnostic

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

token dropping — are content-blind. A uniform sampler treats a critical contact event identically to a static-table frame, and patch-level dropping that ignores temporal saliency removes the global layout that timing- and counting-style tasks need. Both perceptual variants degrade steeply as the memory budget shrinks, leaving open the question of whether the perceptual family is structurally limited or only its existing representatives are.

Robot videos have two complementary kinds of structure: a small number of moments that genuinely matter (object pickup, contact, release, viewpoint shift), and the relatively predictable motion that fills the gaps between them. We argue the right perceptual memory should mirror that structure: keep semantically chosen keyframes for *what was seen and when*, and learn a compact summary for *how things moved between frames*. Our motion summary is trained, JEPA-style [Assran et al., 2023], to predict the latent SigLIP tokens of a future frame from those of the current frame and a learnable motion code, so a single motion token is forced to encode exactly the bridge information that any uniform-subsampling scheme throws away.

We instantiate this idea in three pieces. First, a deterministic CLIP-derivative keyframe detector selects frames at semantic transitions; a delta-based per-keyframe TokenDrop then keeps only patches that changed relative to the previous keyframe, plus their bounding rectangle for spatial contiguity. Second, a small causal-transformer MotionEncoder f_θ runs over fixed-stride short windows and emits one motion token per window; it is pretrained on unlabeled robot video together with an auxiliary MotionDiT decoder that regresses future SigLIP tokens, and the decoder is discarded at deployment. Third, the resulting time-tagged sequence is exposed as a fixed-budget memory state $\mathbf{M}_t$ through the same Memory-as-Context, Memory-as-Modulator, and Memory-as-Expert interfaces used by the MME-VLA suite of Dai et al. [2026] on the $\pi_{0.5}$ backbone. The memory pipeline is task-agnostic and trained once; only the VLA fine-tunes per task. Replacing the perceptual memory module with our keyframe-plus-motion design is therefore a drop-in change at every comparison point, so any observed effect is attributable to memory *content*, not *injection*.

We evaluate this design on RoboMME [Dai et al., 2026] — a long-horizon, history-dependent benchmark spanning temporal, spatial, object, and procedural memory tasks — against uniform-frame-sampling and naive token-drop perceptual baselines at matched memory budgets, under all three integration mechanisms. Our hypothesis is that content-aware keyframes plus a learned motion summary preserve more decision-relevant signal per token than information-blind subsampling, particularly on motion-centric and long-horizon tasks where symbolic memory cannot represent continuous motion. Whether this hypothesis holds across budgets and integration choices is the subject of Sec. 4, where the full quantitative evaluation is currently *in progress*.

Contributions.

- A robot-specialized perceptual memory that combines CLIP-scored semantic keyframes with delta-based TokenDrop and a learned MotionEncoder, decoupling sparse spatial evidence from compact short-range motion summaries inside a fixed token budget.
- A JEPA-style [Assran et al., 2023] latent future-token-prediction objective with an auxiliary MotionDiT decoder that is discarded at deployment, used to pretrain the MotionEncoder on unlabeled robot video so each motion token is forced to encode exactly the information needed to bridge consecutive observations.
- A controlled evaluation protocol on RoboMME [Dai et al., 2026]: our memory plugs into the existing MME-VLA Memory-as-Context, -Modulator, and -Expert interfaces on the $\pi_{0.5}$ backbone, enabling apples-to-apples comparison against uniform-frame-sampling and naive token-drop baselines at matched memory budgets (full quantitative results: *evaluation in progress*).

2 Related Work

Memory-based Robotic Manipulation Models. History-dependent manipulation has motivated a rapidly growing family of memory designs for VLA policies, but the design space remains fragmented. The most direct treatment is to feed history in as additional context: ContextVLA [Jang et al., 2025] concatenates a sliding window of past visual tokens to the transformer input, while BC-RNN [Mandlekar et al., 2021] rolls history through a recurrent state. Both extremes scale poorly —

the former pays linear attention cost in horizon length, the latter compresses without selection. Recent state-space policies, including RoboMamba [Liu et al., 2024] and MITL [Zhou et al., 2025], replace the RNN with Mamba-style updates [Gu and Dao, 2024], preserving constant-memory recurrence while extending the effective context window.

A second line of work inserts an explicit *memory bank* between the encoder and the policy and reads from it on demand. Hamlet [Koo et al., 2025] and MemoryVLA [Shi et al., 2026] cache past visual features together with auxiliary signals such as task instructions or cognitive summaries, and retrieve the relevant entries at each step; SAM2Act [Fang et al., 2025] attaches a similar memory module to a video segmentation backbone so that object identity and action heatmaps remain stable across long horizons. MEM [Physical Intelligence, 2026a] pushes this further by organizing embodied features at multiple temporal resolutions, while BPP [Sobol Mark et al., 2026] recasts long-context imitation learning as the problem of selecting a small set of behaviorally relevant key history frames. HistRISE [Chen et al., 2025] takes the structured-state view to its extreme, tracking object-centric 3D points as a non-differentiable symbolic memory.

A third, increasingly popular direction offloads memory entirely to a large vision-language model. UniVLA [Bu et al., 2025] simply prepends past actions to the prompt, while MemER [Sridhar et al., 2025], Gemini-Robotics-1.5 [Gemini Robotics Team et al., 2025], and the steerable $\pi_{0.7}$ generalist [Physical Intelligence, 2026b] rely on a VLM to produce language-based subgoals or natural-language plans that condition a downstream policy. This keeps the policy itself short-context, but introduces costly modular inference and breaks end-to-end optimization. Taken together, these works establish that memory is essential, yet they differ in backbone, in what is stored, and in how memory is integrated, making head-to-head comparison difficult — the gap our controlled study targets, by training a family of memory-augmented VLAs on the same backbone and evaluating diverse representations and integration mechanisms in a unified setting.

Efficient Video Representation for Robotic Manipulation. Episodic robot videos are heavily redundant: adjacent frames differ only in a few interaction-relevant pixels, and most patches encode static background or the robot's own body. General video understanding addresses this with token-level pruning and resampling — TimeChat-Online [Yao et al., 2025] shows that roughly 80% of visual tokens are naturally redundant in streaming video, and VideoLLM-online [Chen et al., 2024] streams video into a language model under an incremental token budget. In manipulation the constraint is tighter still: a policy must run at control rate while keeping many seconds of history within a fixed token window. Robotic methods therefore aggregate frames at the encoder side — ContextVLA [Jang et al., 2025] amortizes multi-frame context into a compact history bundle, Hamlet [Koo et al., 2025] turns a vanilla VLA into a history-aware policy by switching its inputs at inference time, and long-context diffusion policies [Torne et al., 2025] supervise past-token prediction so that the compressed history retains causally relevant information. A complementary thread asks *which* frames matter: BPP [Sobol Mark et al., 2026] explicitly selects key history frames for imitation learning, and SAM2Act [Fang et al., 2025] reuses a video segmentation foundation model to anchor object identity across long horizons. We build on these observations by combining TokenDropped keyframe patches with a MotionEncoder summary, decoupling sparse visual evidence from high-frequency motion structure and giving every memory variant the same fixed-size video representation budget.

3 Method

3.1 Model Architecture

Our policy is a memory-augmented vision-language-action (VLA) model built on top of the $\pi_{0.5}$ backbone. We initialize from the pre-trained backbone and fine-tune its experts end-to-end in Stage 2 (Sec. **??**). The only components kept frozen throughout both stages are the SigLIP2 visual encoder and, after Stage 1, the MotionEncoder f_θ. We inject the per-episode memory sequence assembled in Sec. 3.2 as additional context tokens.

$\pi_{0.5}$ backbone. At history timestep t, the policy receives language tokens $\boldsymbol{\ell} = \mathrm{Tok}_{\mathrm{text}}(l) \in \mathbb{R}^{N_\ell \times d}$ and multi-view image tokens $\mathbf{o}_t = \mathrm{Tok}_{\mathrm{img}}(I_t) \in \mathbb{R}^{N_o \times d}$ extracted by a frozen SigLIP2 encoder. $\pi_{0.5}$ follows the π_0 multi-expert design and is composed of (i) a *VLM expert* E_{vlm} that fuses language and vision and (ii) an *action expert* E_{act} that predicts an action chunk conditioned on the VLM features

and a denoising timestep τ. Both experts are L-layer transformers connected by a *blockwise causal attention* operator

$$\mathbf{y} \;=\; \mathrm{BlockAttn}(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_G), \tag{1}$$

in which attention is bidirectional within each block $\mathbf{x}_i$ and causal across blocks (block i may attend blocks $1{:}i$ but not future blocks). The action expert is conditioned on τ via AdaLN-Zero modulation: with action latent $\mathbf{s}_t^k$ at layer k,

$$\tilde{\mathbf{s}}_t^k = \mathbf{s}_t^{k-1} + g_\tau \odot \mathrm{BlockAttn}^k\big(\mathrm{Norm}^k_{\mathrm{vlm}}(\mathbf{u}_t^{k-1}),\, \mathrm{Norm}^k_{\mathrm{act}}(\mathbf{s}_t^{k-1}, \tau)\big), \quad \mathbf{s}_t^k = \tilde{\mathbf{s}}_t^k + \tilde{g}_\tau \odot \mathrm{MLP}^k_{\mathrm{act}}\big(\mathrm{Norm}^k_{\mathrm{act}}(\tilde{\mathbf{s}}_t^k, \tau)\big), \tag{2}$$

where $g_\tau, \tilde{g}_\tau$ are gates conditioned on τ. Unlike the original $\pi_{0.5}$, which uses the VLM expert to also generate low-level language commands, we keep the VLM as a pure feature extractor; this leaves a clean interface at which memory can be injected.

Memory token block. To enable history-dependent reasoning, we augment the policy with an explicit memory state $\mathbf{M}_t$:

$$a_t \;\sim\; \pi(\cdot \mid l,\; \mathbf{o}_t,\; \mathbf{M}_t)\,, \tag{3}$$

where $\mathbf{M}_t$ is the length-B projection of the causal prefix $\mathcal{M}_t$ (Eq. 15), containing only observations and motion windows that have fully elapsed before step t. For fair comparison across designs, we allocate a fixed memory token budget B for all memory-based models: each representation produces at most B tokens; if fewer are available, the sequence is zero-padded. Thus all memory variants share the same interface

$$\mathbf{M}_t \;\in\; \mathbb{R}^{B \times d}. \tag{4}$$

We expose the memory sequence $\mathcal{M}$ produced in Sec. 3.2 to the backbone as this token block. The two-token-kind structure of $\mathcal{M}$ — TokenDropped keyframe patches and MotionEncoder motion tokens — is opaque to the backbone: from the policy's perspective $\mathbf{M}_t$ is just a length-B sequence of d-dimensional vectors carrying time tags.

Three integration mechanisms. We systematically study three places at which $\mathbf{M}_t$ can enter the backbone, trading parameter count, attention budget, and isolation of the pretrained VLM features against each other.

(1) Memory-as-Context (MaC). The simplest variant: concatenate $\mathbf{M}_t$ with the standard inputs of the VLM expert,

$$\mathbf{u}_t^0 \;=\; [\,\mathbf{M}_t;\, \mathbf{o}_t;\, \boldsymbol{\ell}\,], \qquad \mathbf{u}_t \;=\; E_{\mathrm{vlm}}(\mathbf{u}_t^0). \tag{5}$$

MaC introduces *no* new parameters and reuses the existing blockwise causal attention (1); its cost is that memory tokens compete with raw observation tokens for the same attention budget inside E_{vlm}, and any noise in $\mathbf{M}_t$ propagates through every downstream feature.

(2) Memory-as-Modulator (MaM). Instead of feeding memory through the VLM, we use it to *modulate* the action expert. At each layer k, the action latent $\tilde{\mathbf{s}}_t^k$ first cross-attends $\mathbf{M}_t$, and the result is mapped to per-feature scale/shift parameters via a zero-initialized MLP that is then composed with AdaLN:

$$\mathbf{r}_t^k = \mathrm{Attn}^k_{\mathrm{mod}}\big(Q{=}\tilde{\mathbf{s}}_t^k,\; K{=}\mathbf{M}_t,\; V{=}\mathbf{M}_t\big), \tag{6}$$

$$(\gamma_t^k, \beta_t^k) = \mathrm{MLP}^k_{\mathrm{mod}}(\mathbf{r}_t^k), \tag{7}$$

$$\widehat{\mathbf{s}}_t^k = \gamma_t^k \odot \mathrm{Norm}^k_{\mathrm{mod}}(\tilde{\mathbf{s}}_t^k) + \beta_t^k, \tag{8}$$

$$\mathbf{s}_t^k = \widehat{\mathbf{s}}_t^k + \tilde{g}_\tau \odot \mathrm{MLP}^k_{\mathrm{act}}\big(\mathrm{Norm}^k_{\mathrm{act}}(\widehat{\mathbf{s}}_t^k, \tau)\big). \tag{9}$$

$\mathrm{MLP}^k_{\mathrm{mod}}$ is initialized so that $(\gamma_t^k, \beta_t^k){=}(1, 0)$ at the start of fine-tuning, making the modulation an exact identity at step zero and protecting the pretrained $\pi_{0.5}$ behavior. MaM adds a moderate number of parameters ($\sim$80M, see Sec. 3.4) and, importantly, leaves the VLM expert untouched.

(3) Memory-as-Expert (MaE). We allocate a dedicated third expert E_{mem} that processes $\mathbf{M}_t$ through its own L layers of self-attention plus MLP:

$$\tilde{\mathbf{v}}_t^k = \mathbf{v}_t^{k-1} + \mathrm{BlockAttn}^k\big(\mathrm{Norm}^k_{\mathrm{mem}}(\mathbf{v}_t^{k-1})\big), \quad \mathbf{v}_t^k = \tilde{\mathbf{v}}_t^k + \mathrm{MLP}^k_{\mathrm{mem}}\big(\mathrm{Norm}^k_{\mathrm{mem}}(\tilde{\mathbf{v}}_t^k)\big), \tag{10}$$

with $\mathbf{v}_t^0 = \mathbf{M}_t$. The action expert's update (2) is then extended to attend *both* VLM and memory features as additional context blocks:

$$\tilde{\mathbf{s}}_t^k = \mathbf{s}_t^{k-1} + g_\tau \odot \mathrm{BlockAttn}^k\big(\mathrm{Norm}^k_{\mathrm{mem}}(\mathbf{v}_t^{k-1}),\, \mathrm{Norm}^k_{\mathrm{vlm}}(\mathbf{u}_t^{k-1}),\, \mathrm{Norm}^k_{\mathrm{act}}(\mathbf{s}_t^{k-1}, \tau)\big). \tag{11}$$

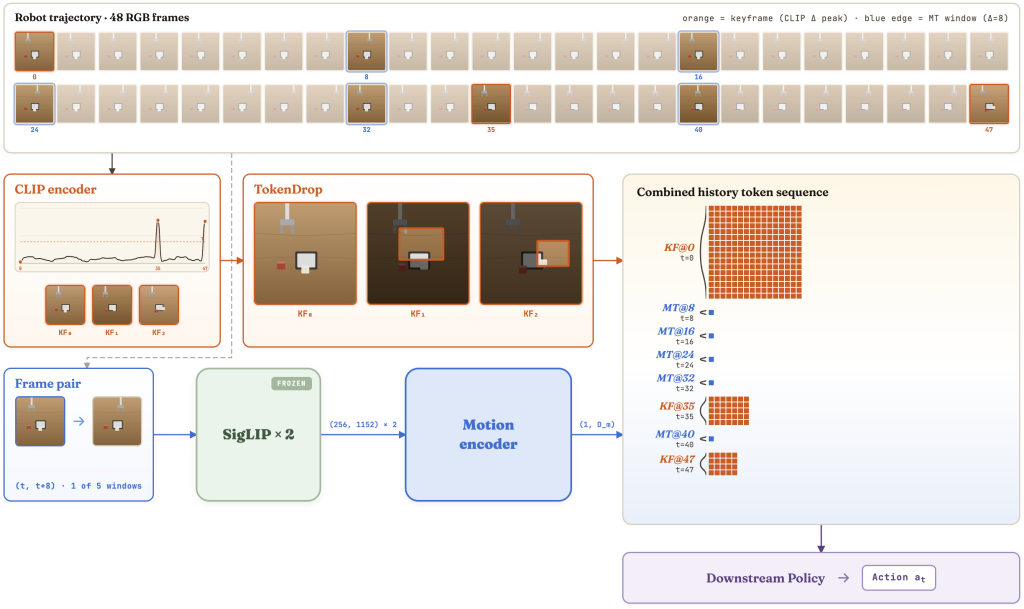

Figure 1: **Overall memory pipeline.** A CLIP encoder selects semantic keyframes from the robot trajectory; TokenDrop retains only the changed patches per keyframe. A learned MotionEncoder compresses each short frame-pair window into a single motion token via JEPA-style latent prediction. The interleaved keyframe and motion tokens form a fixed-budget memory sequence $\mathbf{M}_t$ that conditions the downstream policy to predict action a_t.

The blockwise causal mask is configured asymmetrically: the VLM expert and memory expert each use self-attention only and *do not* attend each other; the action expert is the sole consumer that attends both. This asymmetry preserves the pretrained $\pi_{0.5}$ representations exactly while allocating a separate transformer's worth of capacity ($\sim$190M parameters) to memory processing.

3.2 Memory Token Generation and Training

As the robot executes, the memory $\mathcal{M}_\tau$ grows online from the history of observations accumulated prior to policy step τ. It is a sparse, time-aligned sequence of two complementary token kinds: (i) *keyframe tokens*, which anchor the scene at semantic transitions, and (ii) *motion tokens*, fixed-stride summaries of short-range visual change produced by a learned MotionEncoder. Both kinds are encoded strictly from already-elapsed frames: keyframe tokens are committed as soon as a keyframe is detected, and a motion token m_t over window $[t,\, t{+}\Delta]$ becomes available only after step $t{+}\Delta$ has passed (Eq. 15). All encoding is performed by frozen perception backbones independently of the VLA; no gradient flows back into these encoders during VLA training.

Keyframe detection. Let $\{I_t\}_{t\geq 0} \subset \mathbb{R}^{3\times H\times W}$ denote the sequence of RGB frames observed during execution. Keyframes mark moments of meaningful scene change (object pickup, contact events, viewpoint shifts). Each frame is scored by the smoothed discrete derivative of a frozen CLIP encoder Φ_{CLIP}:

$$d_t \;=\; \left\|\Phi_{\text{CLIP}}(I_t) - \Phi_{\text{CLIP}}(I_{t-1})\right\|_2, \qquad \tilde{d}_t \;=\; (k_{\text{rbf}} * d)_t, \tag{12}$$

where k_{rbf} is a small RBF kernel that suppresses frame-rate jitter. Keyframes are local maxima of $\tilde{d}_t$ exceeding a threshold τ, with the first frame $t = 0$ always retained as an anchor. The detector is purely deterministic preprocessing and has no learnable parameters; τ is a single hyperparameter chosen so that the average inter-keyframe gap is $\sim$32 frames.

Keyframe TokenDrop. Storing the full S-token grid for every keyframe is wasteful: most of the image (background, robot base, table) repeats across keyframes. For every keyframe k_i with $i > 0$ we retain only the patches that actually changed relative to its *previous* keyframe k_{i-1}. Concretely,

on the 16×16 patch grid we mark a patch as *significant* if its mean per-pixel absolute difference between I_{k_i} and $I_{k_{i-1}}$ exceeds δ, and we keep the union of all significant patches together with the patches lying inside their axis-aligned bounding rectangle (the latter preserves spatial contiguity for downstream attention). The first keyframe k_0 is always kept in full (S tokens); each subsequent keyframe contributes $|\mathcal{P}_i| \leq S$ tokens. TokenDrop is also parameter-free.

Motion tokens. Let $\Phi_{\text{SigLIP}} : \mathbb{R}^{3\times H\times W} \to \mathbb{R}^{S\times E}$ ($S = 256$ patches, $E = 1152$) be the frozen SigLIP tokenizer, and let $x_t = \Phi_{\text{SigLIP}}(I_t) \in \mathbb{R}^{S\times E}$; $\{s_t\}$ denotes the proprioceptive state sequence. Independently of keyframe placement, we tile the video with non-overlapping windows of fixed stride Δ (we use $\Delta = 8$). For each window $[t,\ t+\Delta]$ a trainable *MotionEncoder* f_θ compresses the visual change into M latent tokens $m_t \in \mathbb{R}^{M\times D_m}$:

$$m_t \;=\; f_\theta(\, x_t,\ x_{t+\Delta},\ \Delta s_t\,), \qquad \Delta s_t = s_{t+\Delta} - s_t. \tag{13}$$

Internally, f_θ (i) computes the patchwise visual delta $\Delta x = x_{t+\Delta} - x_t \in \mathbb{R}^{S\times E}$, (ii) spatially average-pools Δx to a single global delta token in $\mathbb{R}^{1\times E}$ and projects it to the encoder width H, (iii) projects Δs_t to H and adds it to the pooled delta token (this is the only place proprioception enters the memory pipeline), (iv) appends M learned read-out queries at the tail, and (v) runs a small causal transformer (RMSNorm, 1D-RoPE causal self-attention, SwiGLU FFN) so that each query attends to the fused delta token before it. The last M positions are projected to motion dimension D_m and constitute m_t. Since m_t requires both endpoints of window $[t,\ t+\Delta]$, it becomes causally available only at step $t+\Delta$.

Memory sequence assembly. The final memory for an episode is the time-ordered concatenation

$$\mathcal{M} \;=\; \big[\, x^{\text{full}}_{k_0},\ m_0,\ m_\Delta,\ \ldots,\ x^{\text{TD}}_{k_i},\ \ldots\,\big], \tag{14}$$

where each entry carries its source timestamp as a positional tag. At policy step τ, the policy receives the causally valid prefix

$$\mathcal{M}_\tau \;=\; \big\{\, x^{\cdot}_{k_j} : k_j \leq \tau \,\big\} \;\cup\; \big\{\, m_t : t + \Delta \leq \tau \,\big\}, \tag{15}$$

i.e. keyframe tokens whose frame index is at most τ, and motion tokens whose entire window $[t,\ t{+}\Delta]$ has elapsed by τ. The sequence $\mathcal{M}_\tau$ grows monotonically: at each new step the pipeline appends newly completed tokens without recomputing earlier ones. Keyframe selection and TokenDrop are training-free; the only learnable parameters in the memory pipeline are those of the MotionEncoder f_θ, which is pre-trained in Stage 1 of Sec. 3.3 and then frozen. The trained f_θ, together with the parameter-free keyframe and TokenDrop modules, is what ultimately writes into $\mathcal{M}$.

3.3 Training Objective

The full system has two distinct loss surfaces, optimized in two stages: a latent future-prediction loss $\mathcal{L}_{\text{pred}}$ that trains the MotionEncoder f_θ of Sec. 3.2, and a flow-matching loss $\mathcal{L}_{\text{FM}}$ that trains the VLA backbone together with the integration module.

Latent future-prediction loss for f_θ. We train f_θ jointly with an auxiliary *MotionDiT* decoder g_ϕ: a conditional-diffusion decoder that regresses the SigLIP tokens of the future frame from the current tokens and the motion code. Given $(x_t, x_{t+\Delta})$, we sample a diffusion-style timestep $k \sim \mathcal{U}\{1, \ldots, K\}$ and ask g_ϕ to predict the (token-normalized) future tokens from the current tokens, the motion code, and k:

$$\hat{x}^{(k)}_{t+\Delta} \;=\; g_\phi(\, x_t,\ m_t,\ k\,), \qquad m_t = f_\theta(x_t, x_{t+\Delta}, \Delta s_t), \tag{16}$$

$$\mathcal{L}_{\text{pred}} \;=\; \mathbb{E}_{t,\,k}\Big[\, \big\|\hat{x}^{(k)}_{t+\Delta} \;-\; \text{TokenNorm}(x_{t+\Delta})\big\|_2^2 \,\Big]. \tag{17}$$

TokenNorm is a per-feature standardization applied to the target SigLIP tokens to stabilize the regression target. g_ϕ is a small DiT with AdaLN-zero timestep modulation and a tied gated input/output projection initialized to zero, so at $k = 0$ the decoder acts as a normalized identity copy of x_t and only learns residual corrections driven by m_t. This zero-initialized residual head is the sole reason the loss is stable when f_θ is randomly initialized: gradients flow through g_ϕ into f_θ only via the correction term, forcing m_t to encode exactly the information needed to bridge $x_t \to x_{t+\Delta}$. Crucially, g_ϕ exists only during pre-training: at deployment the robot has already executed both endpoints of every motion window, so memory tokens are produced by a single forward pass through Φ_{SigLIP} and f_θ; no decoding is performed and g_ϕ is discarded.

Flow-matching action loss. Following $\pi_{0.5}$, the policy predicts an H-step action chunk $\mathbf{A}_t = [a_t, a_{t+1}, \ldots, a_{t+H-1}] \in \mathbb{R}^{H\times D}$ via conditional flow matching. Given a ground-truth chunk $\mathbf{A}_t^{\text{data}} \sim p_{\text{data}}$, Gaussian noise $\mathbf{A}_t^{\text{noise}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and a denoising timestep $\tau \sim \mathcal{U}(0,1)$, we form the linear interpolant

$$\mathbf{A}_t^{\tau} \;=\; (1-\tau)\,\mathbf{A}_t^{\text{data}} \;+\; \tau\,\mathbf{A}_t^{\text{noise}}, \tag{18}$$

whose constant time-derivative is $\frac{d}{d\tau}\mathbf{A}_t^{\tau} \;=\; \mathbf{A}_t^{\text{noise}} - \mathbf{A}_t^{\text{data}}$. The action expert parameterizes a conditional velocity field $\mathbf{v}_\theta(\mathbf{A}_t^{\tau}, \tau, \mathbf{h}_t)$, where the context features $\mathbf{h}_t$ are the layer-L outputs of the integrated backbone — so $\mathbf{h}_t$ depends on the chosen integration mechanism: in MaC the memory enters $\mathbf{h}_t$ through the VLM expert (5); in MaM through the per-layer modulation (6–9); in MaE through the cross-expert attention (11). The training objective is

$$\mathcal{L}_{\text{FM}} \;=\; \mathbb{E}_{t,\,\tau,\,\mathbf{A}_t^{\text{data}},\,\mathbf{A}_t^{\text{noise}}}\Big[\,\big\|\mathbf{v}_\theta\big(\mathbf{A}_t^{\tau}, \tau, \mathbf{h}_t\big) \;-\; \big(\mathbf{A}_t^{\text{noise}} - \mathbf{A}_t^{\text{data}}\big)\big\|_2^2\,\Big]. \tag{19}$$

At inference the action chunk is generated by integrating $\mathbf{v}_\theta$ from $\tau = 1$ (pure noise) to $\tau = 0$ (data) through a small number of Euler steps.

Two-stage training. The two losses are optimized sequentially, not jointly, because the memory pipeline encodes a task-agnostic visual-dynamics prior whereas the integration module is task-specific.

Stage 1: Memory-pipeline pre-training. On unlabeled robot videos we jointly train the MotionEncoder f_θ and the auxiliary MotionDiT decoder g_ϕ to minimize $\mathcal{L}_{\text{pred}}$. The SigLIP and CLIP backbones, the keyframe detector, and the TokenDrop module are training-free throughout. After Stage 1, g_ϕ is discarded and f_θ is frozen.

Stage 2: VLA fine-tuning. For every demonstration episode we *pre-compute and cache* the full memory sequence $\mathcal{M}$ using the frozen pipeline of Stage 1. At each training step t we extract the causally valid prefix $\mathcal{M}_t$ (Eq. 15) from the cached sequence and project it to $\mathbf{M}_t \in \mathbb{R}^{B\times d}$; no future motion tokens (i.e. windows with $t' + \Delta > t$) are visible to the policy. SigLIP2 visual tokens are likewise pre-computed and cached, following the $\pi_{0.5}$ recipe. Trainable parameters in Stage 2 are exactly: (i) the VLM expert, (ii) the action expert, and (iii) the integration module — which contributes 0 new parameters for MaC (Eq. 5), $\sim$80M for MaM (Eqs. 6–9), or $\sim$190M for MaE (Eqs. 10–11). The single optimization objective is $\mathcal{L}_{\text{FM}}$. No proprioceptive state enters the backbone in Stage 2 (proprioception only participates in Sec. 3.2 when forming motion tokens).

This decoupling is what makes the same memory pipeline reusable across all three integration variants and across downstream tasks: Stage 1 is run once; Stage 2 is repeated for each (task, integration) configuration.

3.4 Implementation Details

Memory pipeline. SigLIP2 provides $S = 256$ patch tokens of width $E = 1152$ per 256×256 frame. CLIP keyframe detection uses an RBF smoother of width 3 frames and a threshold τ tuned so that the average inter-keyframe gap is $\sim$32 frames. TokenDrop operates on the 16×16 patch grid; a patch is marked significant when its mean per-pixel absolute RGB difference w.r.t. the previous keyframe exceeds $\delta = 10^{-2}$, after which we add the patches inside the axis-aligned bounding rectangle of the significant set. The first keyframe is always retained in full at S tokens. The MotionEncoder uses an 8-frame window at stride $\Delta = 16$, $M = 1$ readout query, an encoder-width $H = 1024$, and a 4-layer causal transformer (RMSNorm, 1D-RoPE, SwiGLU). MotionDiT g_ϕ is a 6-layer DiT with AdaLN-zero modulation and a tied gated input/output projection initialized to zero. All memory-pipeline parameters total $\sim$60M; SigLIP2 and CLIP are frozen.

Memory budget and compression. We cap the keyframe-token memory at $B = 2000$ tokens of width $d = 1024$; the MotionEncoder contributes up to 125 motion tokens separately, giving a combined budget of 2000+125, padding with zeros when shorter. Compared with the naive flattened representation of $T \cdot S$ patch tokens per episode, this is a $\sim 250\times$ reduction at $T \approx 2000$, while preserving both the long-range keyframe anchors (the first keyframe contributes a full $S = 256$ tokens) and short-range dynamics ($M = 1$ motion token per $\Delta = 16$-frame window).

Backbone and training schedule. The base $\pi_{0.5}$ policy processes 576 context tokens (512 visual + 64 language) and contains 3.2B parameters. Stage 2 fine-tuning uses AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$,

weight decay 0), constant learning rate 5×10^{-5} with 5% warmup, gradient-norm clip 1.0, EMA 0.999, action horizon $H=20$, and 80k steps. Effective batch size is 64 for MaC and MaM and 8 for MaE (the larger memory expert is the bottleneck). All runs use $4\times$NVIDIA A40 GPUs with FSDP, $\sim$3–4 days per configuration. Stage 1 uses the same optimizer with a shorter 40k-step schedule on a separate unlabeled video corpus. The SigLIP2 ViT is frozen and its outputs are cached on disk to amortize cost across configurations.

4 Experiments

Our evaluation is structured around three research questions. **RQ1 (memory content).** Under a fixed token budget and a fixed integration mechanism, do content-aware perceptual memories (CLIP keyframes, TokenDrop, learned motion tokens) preserve more decision-relevant signal than the content-blind subsampling baselines used in MME-VLA [Dai et al., 2026]? **RQ2 (budget sensitivity).** How does each memory variant degrade as the memory budget B shrinks — gracefully, or with a cliff? **RQ3 (integration).** Among Memory-as-Context, Memory-as-Modulation, and Memory-as-Expert, which integration mechanism best exploits a given memory representation? At the time of submission, the budget-sensitivity component of RQ2 has finished for both content-blind baselines, while the keyframe-based configurations and the cross-backbone study addressing RQ1 and RQ3 are still running; we report what is currently available and explicitly mark all unfinished cells as *ongoing* (Sec. 4.2).

4.1 Experimental Setup

Benchmark. We evaluate on RoboMME [Dai et al., 2026], a long-horizon, history-dependent manipulation suite that explicitly tests four families of memory: *temporal* (event counting and ordering), *spatial* (layout and viewpoint persistence), *object* (identity under occlusion), and *procedural* (reproducing demonstrated motion). RoboMME provides matched train/eval splits, a unified scoring protocol, and the MME-VLA reference policies, which makes it the natural testbed for a controlled comparison of memory *content* at matched memory budgets.

Baselines and our model. All configurations share the $\pi_{0.5}$ backbone and the *Memory-as-Modulation* (MaM) integration mechanism described in Sec. 3.1, so any difference is attributable to memory content. The full set of compared methods is enumerated in Sec. 4.2 (*Baselines* paragraph): two content-blind baselines (*Even Frame Sampling* and *Even TokenDropping*) swept over five token budgets $B\in\{1024, 512, 256, 128, 64\}$, two keyframe-based ablations (*Clip-Keyframe* as an upper-bound configuration with $\sim$8000+ tokens, and *Clip-Keyframe + TokenDrop* at the matched budget of 2000), our full *Clip-Keyframe + TokenDrop + Motion* memory at 2000+125, and an additional cross-backbone study with OpenVLA and Qwen2.5-VL.

Metric. Following RoboMME [Dai et al., 2026], we report task Success Rate (%) averaged over the benchmark's evaluation episodes; higher is better.

Training and inference. We follow the two-stage recipe of Sec. 3.3: Stage 1 pre-trains the MotionEncoder f_θ once on unlabeled robot video with $\mathcal{L}_{\text{pred}}$, after which f_θ is frozen and reused across every (memory, integration, backbone) configuration; Stage 2 fine-tunes the policy with $\mathcal{L}_{\text{FM}}$ for 80k steps under the optimizer and hardware setup described in Sec. 3.4. At inference the memory pipeline runs strictly causally (Eq. 15): only motion windows whose right endpoint has elapsed before the current step contribute to $\mathbf{M}_t$.

4.2 Main Results

This subsection collects the results currently available against the memory-content question (RQ1) and the budget-sensitivity question (RQ2). The completed runs cover the two content-blind perceptual baselines (*Even Frame Sampling* and *Even TokenDropping*) across the full budget sweep $B\in\{1024, 512, 256, 128, 64\}$ under MaM; the keyframe-based configurations and the cross-backbone study are still running and appear as *ongoing* entries in Tab. 1.

Table 1: Main results on RoboMME. We compare our *Clip-Keyframe* + *TokenDrop* + *Motion* memory against keyframe-only / token-drop variants and the FrameSamp / TokenDrop budget sweeps, all under the *Memory-as-Modulation* integration mechanism. Success Rate is reported in %; *ongoing* denotes runs whose evaluation has not yet finished.

Method	Integration	Memory Budget	Success Rate (%)
MME-VLA Clip-Keyframe + TokenDrop + Motion (Ours)	Modulation	$2000{+}125$	*ongoing*
MME-VLA Clip-Keyframe	Modulation	$8000{+}$	*ongoing*
MME-VLA Clip-Keyframe + TokenDrop	Modulation	2000	*ongoing*
MME-VLA Even Frame Sampling	Modulation	1024	45.87
	Modulation	512	44.51
	Modulation	256	42.90
	Modulation	128	36.54
	Modulation	64	30.42
MME-VLA Even TokenDropping	Modulation	1024	40.17
	Modulation	512	38.02
	Modulation	256	36.36
	Modulation	128	32.76
	Modulation	64	18.11
Other backbones (OpenVLA, Qwen2.5-VL)	—	—	*ongoing*

Baselines. We compare our method against five reference configurations and an extra backbone study, all instantiated under the *Memory-as-Modulation* (MaM) integration mechanism so that differences are attributable to memory *content* rather than *injection*:

- **MME-VLA Clip-Keyframe** retains every 16×16 token of the selected keyframes (§3.2) without compression, giving an upper-bound configuration of $\sim8000+$ tokens.
- **MME-VLA Clip-Keyframe + TokenDrop** adds the temporal-saliency token-dropping policy on top, compressing the memory to a fixed budget of 2000 tokens.
- **MME-VLA Clip-Keyframe + TokenDrop + Motion (Ours)** further appends 125 motion tokens that summarise short-range dynamics, yielding a total budget of $2000{+}125$.
- **MME-VLA Even Frame Sampling** is the naive frame-sampling baseline (§3.2, FrameSamp): uniformly spaced frames with pooled tokens, swept over budgets $\{1024, 512, 256, 128, 64\}$ to characterise how purely temporal subsampling degrades with shrinking memory.
- **MME-VLA Even TokenDropping** is the naive token-dropping baseline that ignores temporal-saliency scoring, swept over the same five budgets to isolate the value of the importance-weighted selection used by our method.
- **Other backbones (OpenVLA, Qwen2.5-VL)** swap the $\pi_{0.5}$ backbone for two alternative VLA backbones with the same memory pipeline, probing how transferable the proposed memory design is across model families.

Findings on the content-blind baselines. Two observations follow directly from the completed runs in Tab. 1. *(i) Frame sampling consistently dominates token dropping at matched budget.* At every budget B, *Even Frame Sampling* outperforms *Even TokenDropping*: the gap is $+5.7$ pt at $B = 1024$ and widens to $+12.3$ pt at $B = 64$. Keeping a few full frames is therefore a strictly stronger memory than retaining the same number of arbitrarily selected patches, even when the patch budget is comparatively generous. *(ii) The two baselines degrade differently as the budget shrinks. Even Frame Sampling* loses 15.45 pt across the full sweep ($45.87 \rightarrow 30.42$), with monotone but accelerating per-halving drops (1.36, 1.61, 6.36, 6.12 pt); *Even TokenDropping* loses 22.06 pt across the same sweep ($40.17 \rightarrow 18.11$), and exhibits a clear cliff between $B = 128$ and $B = 64$ ($32.76 \rightarrow 18.11$, a 14.65 pt single-step drop). This asymmetry is consistent with our motivation (Sec. 1): saliency-agnostic patch removal eventually deletes patches that anchor global layout, after which the policy cannot reconstruct the scene from what remains, while uniform frame subsampling loses temporal resolution more gracefully because each retained frame still carries a complete spatial layout. Whether content-aware selection (CLIP keyframes, TokenDrop, motion tokens) closes this

gap and surpasses *Even Frame Sampling* at matched budgets is the central remaining question; those rows are still running and will replace the *ongoing* cells in Tab. 1.

4.3 Ablation Studies

A full ablation matrix over the components of our memory pipeline and the three integration mechanisms is currently in progress, sharing the same backbone, optimizer, and budget conventions as Sec. 4.2. The planned ablation axes are: (i) keyframe selection (CLIP-derivative vs. uniform) at matched budget; (ii) per-keyframe TokenDrop on/off, isolating the contribution of delta-based patch retention beyond keyframe selection; (iii) MotionEncoder tokens on/off, isolating the contribution of the JEPA-style learned motion summary; (iv) integration mechanism, swapping MaM for Memory-as-Context and Memory-as-Expert (Eqs. 5, 10–11) on the same memory tokens, addressing RQ3; and (v) backbone, swapping $\pi_{0.5}$ for OpenVLA and Qwen2.5-VL with the memory pipeline held fixed, probing transferability. We do not report partial ablation numbers here to avoid over-interpreting unfinished sweeps.

4.4 Analysis

Two qualitatively different degradation modes. Reading the completed entries of Tab. 1 as two budget-success curves makes the structural difference between the baselines explicit. *Even Frame Sampling* stays nearly flat over the looser half of the sweep (1.36 and 1.61 pt drops at $1024 \rightarrow 512$ and $512 \rightarrow 256$) before transitioning into a steeper but still bounded regime in the tighter half (6.36 and 6.12 pt at $256 \rightarrow 128$ and $128 \rightarrow 64$). *Even TokenDropping* starts 5.7 pt below at the loosest budget, tracks comparable per-halving losses through $B = 256$ (2.15, 1.66, and 3.60 pt), and then collapses by 14.65 pt between $B = 128$ and $B = 64$. We read the cliff as evidence that content-blind patch removal crosses a representational threshold: below this budget the surviving patches no longer anchor enough global structure for the action expert to recover the scene, whereas frame-level subsampling continues to deliver intact-but-coarser temporal slices. From a memory-design perspective, this is the empirical signature that motivates content-aware selection — deciding *which* patches to keep is most consequential exactly where the budget is tightest, which is also where motion-centric and counting-style RoboMME tasks place their hardest queries.

Caveats. The analysis above is restricted to the two completed sweeps; we do not extrapolate to the keyframe-based or cross-backbone configurations whose runs are still in progress. We will revisit the per-task breakdown (temporal / spatial / object / procedural) and report failure cases in the final version, once the *ongoing* cells of Tab. 1 are filled in.

5 Conclusion

We presented a robot-specialized perceptual memory that decouples *what was seen and when* from *how things moved between observations*. CLIP-derivative keyframe detection picks frames at semantic transitions, a delta-based per-keyframe TokenDrop retains only the patches that change relative to the previous keyframe, and a small MotionEncoder f_θ, pretrained JEPA-style with an auxiliary MotionDiT decoder that is discarded at deployment, summarizes each short window into a single motion token. The resulting fixed-budget memory is exposed to the $\pi_{0.5}$ backbone through three interchangeable interfaces — Memory-as-Context, Memory-as-Modulation, and Memory-as-Expert — so that content-blind subsampling baselines and our content-aware design can be compared head-to-head on RoboMME at matched memory budgets and matched integration. The completed budget sweeps already establish that the two content-blind perceptual variants of MME-VLA degrade in qualitatively different ways: *Even Frame Sampling* loses 15.45 pt nearly linearly across $B \in [1024, 64]$, while *Even TokenDropping* loses 22.06 pt with a sharp 14.65-pt cliff at $B = 64$, indicating that which patches are retained matters most precisely when the budget is smallest. This is the controlled signature our keyframe-and-motion design is meant to exploit.

Limitations and future work. Our quantitative evaluation is incomplete: at the time of submission, the *Clip-Keyframe*, *Clip-Keyframe + TokenDrop*, and *Clip-Keyframe + TokenDrop + Motion (Ours)* configurations under MaM, the comparison across the three integration mechanisms (MaC / MaM / MaE) called for by RQ3, and the cross-backbone study with OpenVLA and Qwen2.5-VL are all

still running. The analysis we present here is therefore restricted to the two completed content-blind baselines; a per-task breakdown across RoboMME's temporal / spatial / object / procedural categories, formal ablations of TokenDrop and motion tokens, and qualitative failure cases will be reported once those runs finish. Beyond completing the present evaluation, three directions look particularly worth pursuing: (i) online or streaming updates to $\mathbf{M}_t$ that recompute keyframe assignments as the episode evolves; (ii) hybrid memories that combine our perceptual tokens with the language-subgoal memories used by recent VLM-driven policies [Sridhar et al., 2025, Gemini Robotics Team et al., 2025], since the two families capture complementary aspects of history; and (iii) validation on real hardware beyond the RoboMME simulation suite, where sensor noise and timing jitter will further stress the keyframe detector and the MotionEncoder.

References

M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. Learning to act anywhere with task-centric latent actions. In *Robotics: Science and Systems (RSS)*, 2025.

J. Chen, Z. Lv, S. Wu, K. Q. Lin, C. Song, D. Gao, J.-W. Liu, Z. Gao, D. Mao, and M. Z. Shou. VideoLLM-online: Online video large language model for streaming video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18407–18418, 2024.

J. Chen, H. Fang, C. Wang, S. Wang, and C. Lu. History-aware visuomotor policy learning via point tracking. *arXiv preprint arXiv:2509.17141*, 2025.

Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. RoboMME: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.

H. Fang, M. Grotz, W. Pumacay, Y. R. Wang, D. Fox, R. Krishna, and J. Duan. Sam2Act: Integrating visual foundation model with a memory architecture for robotic manipulation. In *International Conference on Machine Learning (ICML)*, 2025.

Gemini Robotics Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. Gonzalez Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.

A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling (COLM)*, 2024.

H. Jang, S. Yu, H. Kwon, H. Jeon, Y. Seo, and J. Shin. ContextVLA: Vision-language-action model with amortized multi-frame context. *arXiv preprint arXiv:2510.04246*, 2025.

M. Koo, D. Choi, T. Kim, K. Lee, C. Kim, Y. Seo, and J. Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. *arXiv preprint arXiv:2510.00695*, 2025.

J. Liu, M. Liu, Z. Wang, P. An, X. Li, K. Zhou, S. Yang, R. Zhang, Y. Guo, and S. Zhang. RoboMamba: Efficient vision-language-action model for robotic reasoning and manipulation. *Advances in Neural Information Processing Systems*, 37:40085–40110, 2024.

A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.

Physical Intelligence. MEM: Multi-scale embodied memory for vision language action models, 2026a.

Physical Intelligence. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities, 2026b.

H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang. MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2026.

N. Sobol Mark et al. BPP: Long-context robot imitation learning by focusing on key history frames, 2026.

A. Sridhar, J. Pan, S. Sharma, and C. Finn. MemER: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.

M. Torne, A. Tang, Y. Liu, and C. Finn. Learning long-context diffusion policies via past-token prediction. *arXiv preprint arXiv:2505.09561*, 2025.

L. Yao, Y. Li, Y. Wei, L. Li, S. Ren, Y. Liu, K. Ouyang, L. Wang, S. Li, S. Li, et al. TimeChat-Online: 80% visual tokens are naturally redundant in streaming videos. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 10807–10816, 2025.

Y. Zhou, Y. Lin, F. Peng, J. Chen, K. Huang, H. Yang, and Z. Yin. MTIL: Encoding full history with mamba for temporal imitation learning. *IEEE Robotics and Automation Letters*, 2025.